

AI Agents for Image and Video Generation

Prompt Engineering for
Video Generation



Video Prompt Attributes

- **Subject:** The "who" or "what" of your video
- **Action:** Describe movements, interactions, etc.
- **Scene:** The "where" and "when" of your video
- **Style:** The video's artistic filter
- **Temporal elements:** To imply changes in time

Video Prompt Attributes

Camera Components

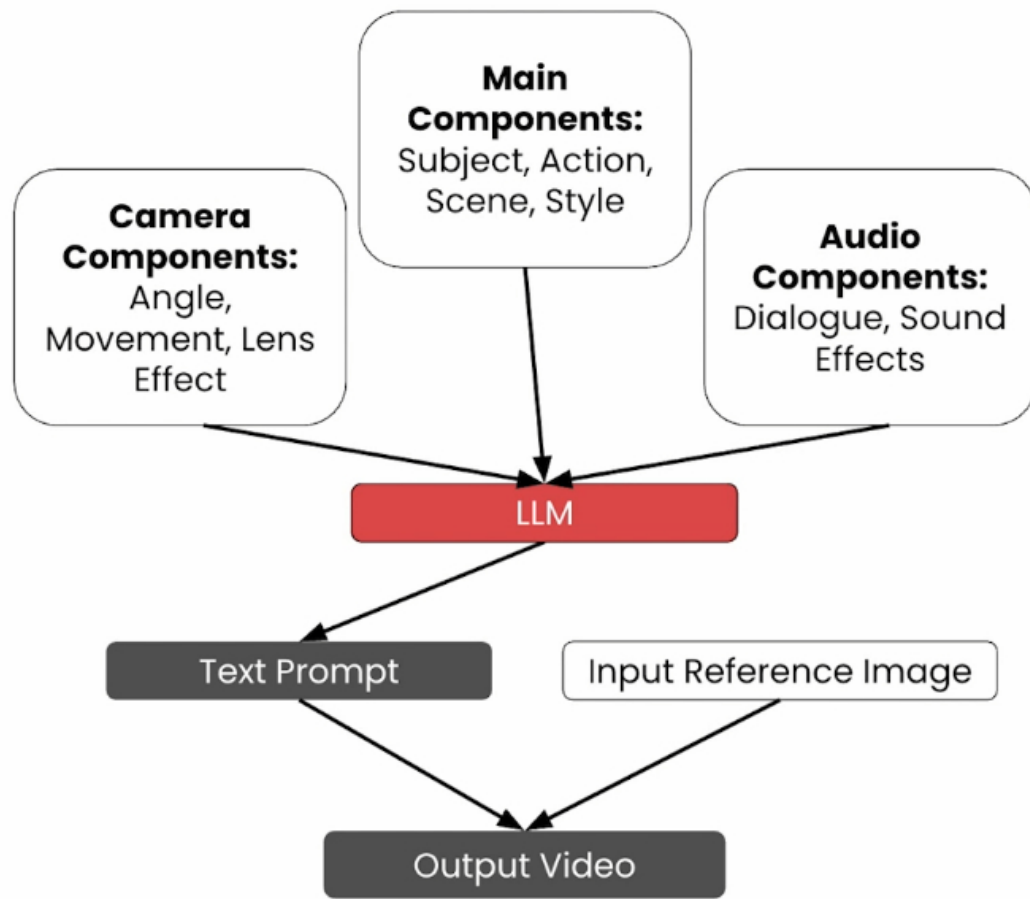
- **Camera angles:** The shot's viewpoint
 - Ex: Eye-Level Shot, High-Angle Shot, Close-Up
- **Camera movements:** For a more cinematic/dynamic experience
 - Ex: Static Shot, Pan (Left/Right), Tilt (Up/Down)
- **Lens effects:** How the camera "sees" the world
 - Ex: Wide-Angle Lens, Lens Flare, Bokeh

Video Prompt Attributes

Audio Components

- **Dialogue:** Spoken words from a character
 - Ex: "Where were you last night?"
- **Sound Effects:** Distinct sounds that occur within the scene
 - Ex: A Ticking Clock, Phone Ringing, Water Splashing

Video Prompts



In the lab

1

Generate video from a basic text prompt

2

Formalize prompt structure

3

Reference input image for style alignment

4

Generate a video from an enhanced prompt

L3: Prompt Engineering for Video Generation

```
import time
from IPython.display import Image, Markdown, display, Video
from google import genai
from google.genai import types
```

```
import os
from helper import authenticate

credentials, project_id = authenticate()
client = genai.Client(
    project=project_id,
    location="global",
    credentials=credentials,
    http_options=types.HttpOptions(
        base_url=os.getenv("GOOGLE_VERTEX_BASE_URL")
    )
)
```

```
IMAGE_MODEL_ID = "gemini-3.1-flash-image-preview"
TEXT_MODEL_ID = "gemini-3-flash-preview"
VIDEO_MODEL_ID = "veo-3.1-fast-generate-001"
```

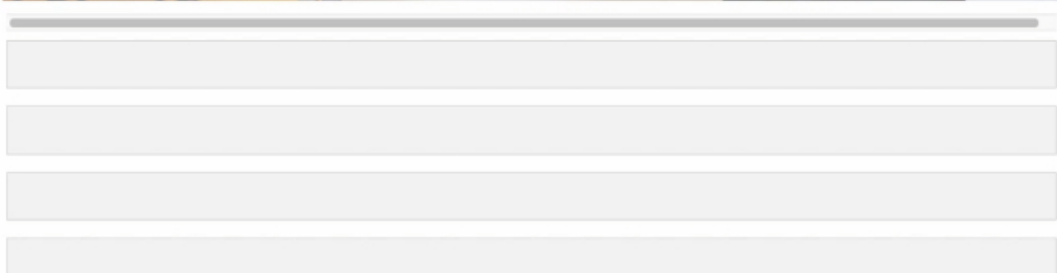
```
def show_video(video, filename):
    with open(filename, "wb") as out_file:
        out_file.write(video)
    display(Video(filename, embed=True, width=600))
```



Simple prompt

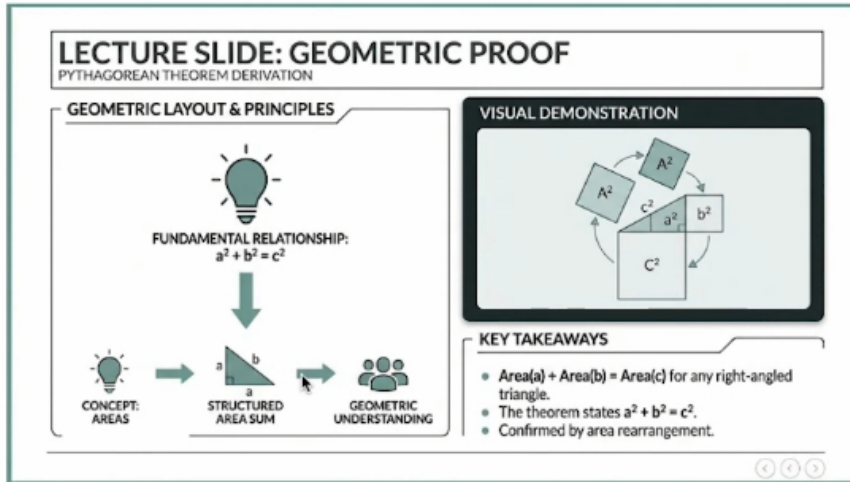
```
operation = client.models.generate_videos(  
    model=VIDEO_MODEL_ID,  
    prompt="A person explaining the Pythagorean theorem.",  
    config=types.GenerateVideosConfig(  
        aspect_ratio="16:9",  
        number_of_videos=1,  
        duration_seconds=8,  
        resolution="720p",  
        person_generation="allow_adult",  
        generate_audio=True,  
    ),  
)  
  
while not operation.done:  
    time.sleep(15)  
    operation = client.operations.get(operation)  
    print(operation)  
  
if operation.response:  
    show_video(operation.result.generated_videos[0].video.video_bytes,  
                "original.mp4")
```



A woman with dark hair and glasses, wearing a dark blazer over a white shirt, stands in a classroom. She is pointing with a thin stick at a whiteboard. The whiteboard features a right-angled triangle with vertices labeled 'a', 'b', and 'c' at the corners. The sides are labeled 'a2', 'b2', and 'c2'. Above the triangle, the equation $a + b2 = c2$ is written. To the right of the triangle, the equation $b2 = c2$ is written. The background shows rows of empty wooden desks and chairs in a well-lit room with circular ceiling lights.

Generate Input Image

```
display(Image("slide-image.png", width=500))
```



```
with open("slide-image.png", "rb") as f:
    image = f.read()

response = client.models.generate_content(
    model=IMAGE_MODEL_ID,
    contents=[
        types.Part.from_bytes(
            data=image,
            mime_type="image/png",
        ),
        """Generate a professor in a classroom full of students in
        front of the provided slide image."""
    ],
    config=types.GenerateContentConfig(
        response_modalities=['IMAGE'],
        image_config=types.ImageConfig(
            aspect_ratio="16:9",
        ),
    ),
)

for part in response.candidates[0].content.parts:
    if part.inline_data:
        image_data=part.inline_data.data
        display(Image(data=image_data, width=500))

with open("video-image.png", "wb") as image_file:
    image_file.write(image_data)
```



1x Auto



```
with open("slide-image.png", "rb") as f:
    image = f.read()

response = client.models.generate_content(
    model=IMAGE_MODEL_ID,
    contents=[
        types.Part.from_bytes(
            data=image,
            mime_type="image/png",
        ),
        """Generate a professor in a classroom full of students in
        front of the provided slide image."""
    ],
    config=types.GenerateContentConfig(
        response_modalities=['IMAGE'],
        image_config=types.ImageConfig(
            aspect_ratio="16:9",
        ),
    ),
)

for part in response.candidates[0].content.parts:
    if part.inline_data:
        image_data=part.inline_data.data
        display(Image(data=image_data, width=500))

with open("video-image.png", "wb") as image_file:
    image_file.write(image_data)
```

```

        types.Part.from_bytes(
            data=image,
            mime_type="image/png",
        ),
        """Generate a professor in a classroom full of students in
        front of the provided slide image."""
    ],
    config=types.GenerateContentConfig(
        response_modalities=['IMAGE'],
        image_config=types.ImageConfig(
            aspect_ratio="16:9",
        ),
    ),
)

for part in response.candidates[0].content.parts:
    if part.inline_data:
        image_data=part.inline_data.data
        display(Image(data=image_data, width=500))

with open("video-image.png", "wb") as image_file:
    image_file.write(image_data)

```



```

keywords = [subject, action, scene, style, camera_angle,
            camera_movement, sound_effects, dialogue]

gemini_prompt = f"""
Your task is to expand the following keywords into a single, high-fidelity,
descriptive prompt for video generation. Every single keyword MUST be
included. Synthesize them into a single, cohesive, and cinematic
instruction. Do not add any new core concepts. Output ONLY the final
prompt string, without any introduction or explanation. Mandatory Keywords:
{",".join(keywords)}
"""

response = client.models.generate_content(
    model=TEXT_MODEL_ID,
    contents=gemini_prompt,
)

video_prompt = response.text
display(Markdown(response.text))

```

A photorealistic eye-level shot captures a professor in a classroom giving a lecture on the Pythagorean theorem, presenting in front of students while the camera performs a slow zoom in; the professor explains, "When you square a side length, like a or b, you are literally calculating the area of a square built perfectly off that side," accompanied by the faint clicking of computer keys.

```
operation = client.models.generate_videos(  
    model=VIDEO_MODEL_ID,  
    prompt=video_prompt,  
    image=types.Image.from_file(location="video-image.png"),  
    config=types.GenerateVideosConfig(  
        aspect_ratio="16:9",  
        number_of_videos=1,  
        duration_seconds=8,  
        resolution="720p",  
        person_generation="allow_adult",  
        generate_audio=True,  
    ),  
)  
  
while not operation.done:  
    time.sleep(15)  
    operation = client.operations.get(operation)  
    print(operation)  
  
if operation.response:  
    show_video(operation.result.generated_videos[0].video.video_bytes,  
                "enhanced.mp4")
```



```

00\x00Dduuid\xd8\xfe\xc3\xd6\x1b\x0eH<\x92\x97X(\x87~\xc4\x81\x00\x00\x00\x00
manifest\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00D7jumb\x00\x00\x00\x1ejum
dc2pa...'
)
),
],
rai_media_filtered_count=0
) result=GenerateVideosResponse(
generated_videos=[
GeneratedVideo(
video=Video(
mime_type='video/mp4',
video_bytes=b'\x00\x00\x00 ftypisom\x00\x00\x02\x00isomiso2avc1mp41\x
00\x00Dduuid\xd8\xfe\xc3\xd6\x1b\x0eH<\x92\x97X(\x87~\xc4\x81\x00\x00\x00\x00
manifest\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00D7jumb\x00\x00\x00\x1ejum
dc2pa...'
)
),
],
rai_media_filtered_count=0
)

```

